

AGS Workforce — Employee Scheduling Portal

Teammate Handover Manual

Prepared: 2026-04-15 **Owner (handing off):** Jaswanth Iboyapati **Project codename:** AGS Workforce / Employee Scheduling Portal

1. What this app is (at a glance)

AGS Workforce is an internal **Workforce Management Portal** for AMGSOL. It replaces spreadsheets and ad-hoc group chats for:

- Weekly **shift scheduling** per client (Ameresco, Cleanleaf, Standard Solar, Puresky).
- **Clock-in / clock-out** with live shift status.
- **Leave management** (CL / SL / Comp / LOP) with a Team Lead → Manager → Admin approval chain.
- **Food coupons** (dinner tracking with Wed ₹160 / other-day ₹120 pricing).
- **Cab drops, asset management, budgeting, celebrations (birthdays/anniversaries), HR reports, ideas board**, and an **AI assistant** for natural-language queries over the DB.

It is a **full-stack production-ready app** (not a prototype). Frontend = React + Vite + Tailwind. Backend = Node.js + Express + PostgreSQL (Supabase). Deployed on **Vercel**.

2. Roles & permission model

Role	Typical user	What they can do
admin	CEO / Co-founders (e.g. Dileep Siriki)	Full access. Final leave approver. Manages users, clients, prices.
manager	Narsimha Karthik, Dileep Siriki	Second-level leave approver. Sees all teams. HR reports.
team_lead	Sanjay Gunde (Ameresco), Arun Pandian (Cleanleaf), Srinivasa Krishnan (Standard Solar + Puresky)	First-level leave approver. Uploads schedules for their client. Sees only their team.
employee	Everyone else	Clock in/out. Request leaves. View own schedule / leaves / comp-offs.

Approval chain: Employee → Team Lead → Manager → Admin → **approved** .

3. Repository layout

```
Employee Scheduling Portal/
├─ src/                                # React frontend
│   ├── App.jsx                        # Root component; routing, auth state, nav
│   ├── main.jsx                       # React entrypoint
│   ├── index.css                      # Tailwind + globals
│   └─ api/
│       ├── client.js                  # fetch wrapper, JWT storage, 401 handling
│       └─ normalize.js                # API-payload → UI-shape adapters
│   ├── components/                    # All React views/modals (see §5)
│   ├── data/mockData.js               # Static seed data (clients, icons, role labels)
│   ├── hooks/useModalKeyboard.js
│   └─ utils/ssoAutoLogin.js           # Auto-login from AGS Suite parent app
├─ backend/                            # Node/Express API
│   ├── src/
│   │   ├── index.js                  # App bootstrap, CORS, route mounting
│   │   ├── db/
│   │   │   ├── pool.js               # pg connection pool
│   │   │   └─ migrations/001_initial_schema.sql
│   │   ├── lib/db.js                 # query helpers
│   │   ├── middleware/
│   │   │   ├── auth.js               # JWT verify + requireRole()
│   │   │   └─ error.js               # error handler
│   │   └─ routes/                    # One file per resource (see §6)
│   ├── scripts/                       # seed-admin, seed-clients, seed-employees-from-sh
│   ├── package.json
│   └─ vercel.json
├─ docs/                               # Schema, setup, data seeds
│   ├── supabase_schema.sql            # Full DDL (use this in Supabase SQL edito
│   ├── DATABASE_PLAN.md               # Schema & indexes rationale
│   ├── SUPABASE_SETUP.md              # Supabase project setup
│   ├── EMPLOYEE_DATA_SCHEMA.md         # Shape of employee seed data
│   ├── SEED_EMPLOYEES_FROM_SHEET.md    # How to seed from the spreadsheet
│   ├── SCHEDULE_UPLOAD_GUIDE.md        # CSV format for schedule uploads
│   ├── DATA_AND_SEEDS_README.md
│   ├── INSERT_RECORDS_FROM_FRONTEND.md
│   ├── NEW_FEATURES.md
│   ├── SCHEMA_IMPROVEMENTS.md
│   ├── SEED_EXAMPLE.sql
│   ├── employee-data.json             # Employee seed JSON
│   └─ clients-to-seed.json
├─ public/                             # Favicon + left/right panel logos
├─ scripts/import_schedule.py           # Python helper for bulk schedule imports
└─ index.html
```

```
├─ package.json           # Frontend deps (react, vite, tailwind)
├─ vite.config.js
├─ tailwind.config.js
├─ postcss.config.js
├─ vercel.json           # Vercel routing (frontend + API)
├─ .env.example
├─ .gitignore
└─ README.md
```

4. Tech stack

Frontend

- React 18 + Vite 5 + Tailwind 3
- No routing library — single-page with state-driven view switching in `App.jsx`
- JWT stored in `localStorage` ; auto-logout on 401

Backend

- Node 18+ ESM, Express 4
- PostgreSQL via `pg` (Supabase connection string works out of the box)
- Auth: `bcryptjs` + `jsonwebtoken` (7-day expiry, configurable via `JWT_EXPIRES_IN`)
- Validation: `zod`
- AI assistant: `@google/generative-ai` (Gemini)

Infra

- Deployed as two Vercel projects (frontend static + backend serverless) OR single service with `SERVE_STATIC=true`
- Database: Supabase Postgres

5. Frontend components — what each file does

All under `src/components/` . Read this like a table of contents for the UI.

Shell / layout

File	Purpose
<code>Header.jsx</code>	Top bar: user menu, notifications bell, changelog.
<code>RightPanel.jsx</code>	Branded right-side decorative panel (logo).
<code>FilterBar.jsx</code>	Week / Client / Search filters on shift views.

File	Purpose
<code>Toast.jsx</code>	Global toast notifications.
<code>ErrorBoundary.jsx</code>	Catches render errors and shows fallback UI.
<code>RoleBadge.jsx</code>	Colored pill for admin/manager/lead/employee.
<code>DateRangeFilter.jsx</code>	Small shared date-range picker.

Auth

File	Purpose
<code>LoginPage.jsx</code>	Email/password login, also handles SSO handoff.

Dashboard & shift views

File	Purpose
<code>DashboardView.jsx</code>	Main landing page. KPIs: on-shift-now, on-leave-today, upcoming birthdays, quick actions. Uses <code>actualToday</code> (real calendar date).
<code>ShiftsTable.jsx</code>	The central table: Employee / Shift Time / Status / Clock-in / Clock-out. Colour codes green = logged in, red = not logged in, grey = off-shift.
<code>MyWorkView.jsx</code>	Employee's personal shift card with Clock In / Clock Out button.
<code>ScheduleGridView.jsx</code>	Read-only week-grid view of schedules.
<code>ScheduleBuildGrid.jsx</code>	Interactive grid team leads use to build a weekly schedule.
<code>SchedulesView.jsx</code>	Wrapper: list uploaded schedules, open build grid, view grid, delete.
<code>UploadSchedules.jsx</code>	CSV upload flow for team leads (see <code>docs/SCHEDULE_UPLOAD_GUIDE.md</code> for format).
<code>AttendanceCalendar.jsx</code>	Per-employee monthly attendance calendar with leave/shift markers.

People / team

File	Purpose
<code>EmployeeView.jsx</code>	Employee self-service: my leaves, request leave form (CL/SL/Comp/LOP), my requests + cancel.
<code>EmployeeModal.jsx</code>	Click any employee → pop-up with leave balances and quick request form.
<code>TeamView.jsx</code>	Team lead / manager view of their direct reports.
<code>UserManagementView.jsx</code>	Admin: list all users, change roles, link team-lead ↔ manager, deactivate.

File	Purpose
UserManagementModal.jsx	Modal variant of the above for quick edits.
EditUserModal.jsx	Edit a single user's profile fields.
AddMemberModal.jsx	Add a new employee.
AddClientModal.jsx	Add a new client company.

Leaves

File	Purpose
LeavesView.jsx	The "Leaves" tab. Has sub-tabs: My requests, Pending (for lead/manager), Approved history. Approve/Reject buttons.
LeaveReportView.jsx	Admin/Manager/Lead only. Per-employee leave breakdown (CL used, SL used, comp used, LOP) with sorting, year filter, CSV export.
CompOfffView.jsx	Comp-offs earned/used history; admin can grant comp-off for holiday work.

Notifications

File	Purpose
NotificationsPanel.jsx	Bell-icon drawer. Leave requests, auto-logout notices, shift-change requests, etc. Uses <code>DISTINCT ON (shift_date)</code> under the hood to de-duplicate auto-logout notices.

Food / cab / allowances

File	Purpose
FoodCabView.jsx	Parent tab container for food coupons + cab drops.
DinnerTrackingView.jsx	Food coupons. Daily token marking, monthly ₹ tally (Wed ₹160, others ₹120). Admin can edit prices. CSV export (monthly per-employee + daily breakdown).
CabDropsView.jsx	Track cab drops per employee per day.

Finance / ops

File	Purpose
BudgetingView.jsx	Monthly budget categories, actuals, variance.
ReportsView.jsx	HR/ops reports. Month selector (Jan–Dec), year selector (or Full Year), custom date range, CSV download.

File	Purpose
AssetManagementView.jsx	Laptop/equipment issuance, ownership, return tracking. Largest single component (~1500 LOC).

Culture

File	Purpose
CelebrationsView.jsx	Birthdays & work anniversaries page.
CelebrationBanner.jsx	Dashboard strip showing today's celebrations separately (highlighted).
IdeasView.jsx	Internal ideas/suggestion board with upvotes and comments.

AI / misc

File	Purpose
AIAssistant.jsx	Floating chat widget. Sends NL queries to <code>/api/assistant</code> which translates to read-only SQL via Gemini.
ChangelogModal.jsx	Shows "What's new" release notes.

6. Backend routes — what each file does

All under `backend/src/routes/` . Every route file exports an Express router mounted in `backend/src/index.js` under `/api/<name>` .

File	Base path	Key endpoints	Notes
auth.js	/api/auth	POST /login , POST /logout , POST /sso-token	Email+password → JWT. SSO endpoint used by AGS Suite.
users.js	/api/users	GET /me , GET / (list), PATCH /:id , POST / , DELETE /:id , GET /my-team	Admin/manager can list; employees get only themselves. /my-team scopes to team lead's reports.
clients.js	/api/clients	GET / , POST / , PATCH /:id , DELETE /:id	Ameresco, Cleanleaf, Standard Solar, Puresky.
departments.js	/api/departments	CRUD	Internal departments.
assignments.js	/api/assignments	CRUD	Employee ↔ client assignments.

File	Base path	Key endpoints	Notes
leave-requests.js	/api/leave-requests	POST / , GET / , PATCH /:id/approve , PATCH /:id/reject , DELETE /:id , GET /balance , GET /balance-all	Core leave workflow. balance-all = admin per-employee breakdown (CL/SL/Comp/LOP). Comp leave auto-consumes oldest comp_off row on approval.
schedules.js	/api/schedules	POST / (upload), GET / , DELETE /:id	Body: client_id, iso_year, start_week, weeks_count, rows[] .
shifts.js	/api/shifts	GET / (by week/client), POST /clock-in , POST /clock-out , POST /auto-logout	Largest route file (~1100 LOC). Auto-logout runs 3 h after shift end; uses DISTINCT ON (ce.shift_date) to avoid duplicate notices.
shift-changes.js	/api/shift-changes	Request/approve shift swaps	
assets.js	/api/assets	CRUD + assign/return	Laptops, peripherals.
celebrations.js	/api/celebrations	GET /today , GET /upcoming	/upcoming excludes today (dashboard shows today separately).
allowances.js	/api/allowances	CRUD	Per-employee allowance records.
budgeting.js	/api/budgeting	CRUD	Monthly budget categories.
reports.js	/api/reports	GET /hr , GET /hr.csv	HR rollup, CSV export.
dinners.js	/api/dinners	GET / , POST / , DELETE /:id , GET /summary , GET /settings , PATCH /settings	Food coupons. settings table holds Wed vs regular price.
ideas.js	/api/ideas	CRUD + upvote	Ideas board.
holidays.js	/api/holidays	CRUD	Public holiday calendar. Drives comp-off earning logic.
assistant.js	/api/assistant	POST /query	Gemini → SQL → rows. Largest route (~1270

File	Base path	Key endpoints	Notes
			LOC) because of prompt engineering & safety filters.

Middleware

- `middleware/auth.js` — `authenticate()` (JWT verify, sets `req.user`), `requireRole('admin', 'manager', ...)`.
- `middleware/error.js` — central error handler, hides stack traces in production.

DB layer

- `db/pool.js` — pg Pool singleton; reads `DATABASE_URL`.
- `lib/db.js` — `query()` / `queryOne()` helpers.
- `db/migrations/001_initial_schema.sql` — same DDL as `docs/supabase_schema.sql`.

7. Database schema (summary)

Full DDL: `docs/supabase_schema.sql`. Core tables:

Table	Purpose
<code>users</code>	email, password_hash, role, team_lead_id, manager_id, profile fields
<code>clients</code>	Ameresco / Cleanleaf / Standard Solar / Puresky
<code>departments</code> , <code>assignments</code>	Org structure
<code>schedules</code>	Uploaded weekly schedule headers
<code>schedule_entries</code>	Per-employee per-day shift rows
<code>clock_entries</code>	Actual clock-in / clock-out per shift_date
<code>leave_requests</code>	type (<code>casual</code> , <code>sick</code> , <code>comp</code> , <code>loss_of_pay</code>), status, approver IDs
<code>comp_offs</code>	earned vs used comp-off credits
<code>holidays</code>	Public holidays (drive comp-off earning)
<code>notifications</code>	Bell-icon feed
<code>assets</code> , <code>asset_assignments</code>	Equipment tracking
<code>dinners</code> , <code>food_coupon_settings</code>	Dinner tokens + pricing
<code>cab_drops</code>	Per-day cab log

Table	Purpose
budgets	Monthly budget lines
ideas , idea_votes , idea_comments	Ideas board
celebrations (view)	Derived from users.date_of_birth / joining_date

Leave policy (enforced in `leave-requests.js`):

- 12 Casual Leaves / year (1 per month)
- 4 Sick Leaves / year
- Comp Leaves earned from working on a holiday
- After all 16 paid leaves used → Loss of Pay

8. Setup from scratch

Prerequisites

- Node 18+
- A Supabase project (or any Postgres 14+)
- `DATABASE_URL` connection string

Steps

```
# 1. Unzip and enter project
cd "Employee Scheduling Portal"

# 2. Install frontend deps
npm install

# 3. Install backend deps
cd backend && npm install && cd ..

# 4. Environment files
cp .env.example .env
#   edit .env → set VITE_API_URL (e.g. http://localhost:3000)

# backend env (create backend/.env):
#   DATABASE_URL=postgresql://...
#   JWT_SECRET=<long-random-string>
#   JWT_EXPIRES_IN=7d
#   CORS_ORIGIN=http://localhost:5173
#   GEMINI_API_KEY=<optional, only for AI assistant>
#   SEED_ADMIN_EMAIL=admin@amgsol.com
#   SEED_ADMIN_PASSWORD=admin123
```

```
# 5. Database
#   In Supabase SQL editor, paste docs/supabase_schema.sql and run.
#   (Or: psql $DATABASE_URL -f docs/supabase_schema.sql)

# 6. Seed
cd backend
npm run seed          # creates admin user
npm run seed:clients  # seeds 4 clients
npm run seed:employees # seeds from docs/employee-data.json
cd ..

# 7. Run (both servers together)
npm run dev:all
#   frontend: http://localhost:5173
#   backend:   http://localhost:3000
#   health:    http://localhost:3000/api/health
```

Login after seeding

Use the admin credentials you set in `backend/.env` via `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD`. From there, create/manage other users via the **User Management** view.

9. Deployment (Vercel)

Two project pattern (current):

1. **Frontend project** — root of repo. `vite build` → `dist/`. `vercel.json` at root handles SPA rewrites.
2. **Backend project** — `backend/` folder. `backend/vercel.json` maps all requests to `src/index.js` (serverless). Env vars set in Vercel UI: `DATABASE_URL`, `JWT_SECRET`, `CORS_ORIGIN` (frontend URL), `GEMINI_API_KEY`.

Single-service alternative: set `NODE_ENV=production` and `SERVE_STATIC=true` on the backend, build the frontend, and Express will serve `../dist` (see `backend/src/index.js` lines 68–77).

10. Things to know / gotchas

- **JWT lives in localStorage** — does not sync across devices (expected).
- **Auto-logout buffer** is 3 hours after shift end. Timezone logic is currently IST-based; CST employees may see odd timing.
- **Comp leave validation** happens on request (must have enough earned comp-offs). Consumption happens on approval — oldest comp-off row is marked `used` first.

- **Old leave types** (`annual` , `personal` , `emergency` , ...) from early seed data are auto-mapped to `casual` in balance calculations. Do not re-introduce these types.
- **Food coupon pricing** is in `food_coupon_settings` (row 1). Edit via the portal UI, not the DB, so the audit log captures who changed it.
- **Dashboard date** uses `actualToday` (real wall-clock date), not the overnight-shifted `today` . Do not "fix" this — it was deliberate.
- **.env* files contain secrets.** Before zipping this repo, strip `.env` , `.env.prod` , `.env.vercel` , `backend/.env` .
- **node_modules / dist / .vercel** have been removed. Run `npm install` in both root and `backend/` after unzipping.

11. Key people (domain context)

- **Shree** — needs admin-level access for leave reports.
- **Gautami** — finance; needs max visibility for leave/finance reports.
- **Dileep Siriki** — Manager/CEO level; final approver.
- **Narsimha Karthik** — Manager.
- **Sanjay Gunde** — Ameresco team lead.
- **Arun Pandian** — Cleanleaf team lead.
- **Srinivasa Krishnan** — Standard Solar + Puresky team lead.

12. Where to look first when a bug comes in

Bug area	File to open first
Login / auth / session	<code>backend/src/routes/auth.js</code> , <code>src/components/LoginPage.jsx</code> , <code>src/api/client.js</code>
Wrong leave balance	<code>backend/src/routes/leave-requests.js</code> (balance endpoints), <code>src/components/LeaveReportView.jsx</code>
Wrong clock-in status	<code>backend/src/routes/shifts.js</code> , <code>src/components/ShiftsTable.jsx</code> , <code>src/components/MyWorkView.jsx</code>
Schedule upload issues	<code>backend/src/routes/schedules.js</code> , <code>src/components/UploadSchedules.jsx</code> , <code>docs/SCHEDULE_UPLOAD_GUIDE.md</code>
Food coupon totals	<code>backend/src/routes/dinners.js</code> , <code>src/components/DinnerTrackingView.jsx</code>
Dashboard counts off	<code>src/components/DashboardView.jsx</code> (uses <code>actualToday</code>)

Bug area	File to open first
Duplicate notifications	backend/src/routes/shifts.js (auto-logout DISTINCT ON), src/components/NotificationsPanel.jsx
AI assistant weirdness	backend/src/routes/assistant.js , src/components/AIAssistant.jsx

13. Not shipped in this zip (and why)

Deleted during handover cleanup:

- node_modules/ , backend/node_modules/ → recreate with npm install
- dist/ → recreate with npm run build
- .vercel/ → created fresh by vercel link if redeploying
- .DS_Store → macOS clutter
- Root leftpanel.png , rightpanel.png → duplicates of public/ versions
- Employee data*.xlsx (root + docs/) → raw data already in docs/employee-data.json
- PROTOTYPE_NOTICE.md → outdated (backend now exists)
- docs/PITCH_*.md , docs/WHY_BUILD_EMPTY.md → internal pitch notes, not handover-relevant

Kept: all source, migrations, schema SQL, seeds, schedule CSV templates.

14. Contact

If anything is unclear, ping me (Jaswanth). The fastest orientation path is:

1. Read this doc.
2. npm install && npm run dev:all .
3. Log in as admin → click through every nav item.
4. Open the corresponding component from §5 to see how it's wired.